WEB LAYER HANDOFF

Project Overload v2
Web App Proxy Guide

A clean explanation of every UI proxy route in the web app, what it forwards to, why it exists, and what user context it carries. This is meant as a Wednesday pickup document, so it is written for fast reacquisition rather than code archaeology.

PROXY ROUTES

24

The current web-layer routes that forward through `proxyToApi`.

DIRECT API CLIENT ROUTES

3

Run status, PDF, and HTML are fetched directly instead of using the shared proxy helper.

RECENT FIX

1

The scheduled report status route was added to the proxy layer on March 23, 2026.

Prepared from `apps/web/src/app.ts`

Stored in `docs/`

1. PROXY LAYER BASICS

What the web proxy layer is doing for us

The web app is not only serving HTML. It also exposes a UI-facing API under `/api/*` and forwards many of those calls to the backend API service. This gives us one browser-facing surface while keeping the API base URL, internal API key, and user forwarding behavior in one place.

Why proxy at all?

It hides the raw API host from the browser, centralizes error handling, keeps auth forwarding consistent, and lets the web app shape user-facing failures without duplicating logic in the UI script.

What every proxy shares

The helper builds the target URL from `api_base_url + path`, forwards JSON, adds `x-api-key` when `WEB_INTERNAL_API_KEY` exists, and adds `x-ui-user` when the web cookie identifies a logged-in user.

Shared behavior worth remembering: if the API server cannot be reached, the web layer returns a `502` with a direct message telling us to check whether the API is running. If the API returns non-JSON, the web layer also turns that into a `502` with a retry-oriented message.

User context

Most stateful routes read the `po_user` cookie and pass it through as `x-ui-user`.

Body handling

Any request with a body is serialized as JSON and gets a `content-type: application/json` header.

Allowed methods

The shared helper currently supports `GET`, `POST`, and `PUT`.

```
async function proxyToApi( ... ) {
  const requestHeaders = {
    ... buildApiAuthHeader(),
    ... (additional_headers ?? {})
  };
  if (body !== undefined) requestHeaders["content-type"] = "application/json";
  response = await fetch(api_base_url + path, { method, headers, body: JSON.stringify(body)
});
   ...
}
```

2. CHAT AND SCHEDULING

Routes that support chat memory and scheduled reports

These are the proxies most tied to user-specific state. They nearly all forward the current username, because chat sessions and scheduled reports are meant to be isolated per signed-in user or tenant context.

Chat session proxies

Used by the live chat shell to list prior sessions and save chat-level updates.

2 ROUTES

WEB ROUTE	UPSTREAM API	EXPLANATION	CONTEXT
`GET /api/chat/sessions`	`GET /ui/chat-sessions`	**Lists the saved chat sessions for the current UI user.** This powers the left-side chat history rail.	Requires `po_user`. Forwards `x-ui-user`.
`PUT /api/chat/sessions/:id`	`PUT /ui/chat-sessions/:id`	**Updates a single saved chat session.** Used for renames, persisted state, and similar session-level changes.	Requires `po_user`. Forwards `x-ui-user` and JSON body.

Scheduling proxies

Used after report generation to draft, save, list, inspect, pause, and reactivate scheduled report profiles.

5 ROUTES

WEB ROUTE	UPSTREAM API	EXPLANATION	CONTEXT
`GET /api/runs/:runId/schedule-draft`	`GET /report-runs/:runId/schedule-draft`	**Builds the schedule popup draft from a finished report run.** Returns per-question rerun guidance and defaults for the modal.	Forwards `x-ui-user` when present.

WEB ROUTE	UPSTREAM API	EXPLANATION	CONTEXT
`POST /api/runs/:runId/schedule-profile`	`POST /report-runs/:runId/schedule-profile`	**Saves the actual scheduled report profile.** Stores cadence, timezone, windowing instructions, question execution plan, query template snapshot, and report template.	Forwards JSON body and `x-ui-user`.
`GET /api/scheduled-reports`	`GET /scheduled-reports`	**Lists all scheduled report profiles visible to the UI user.** Powers the left tile grid on Scheduled Reports.	Forwards `x-ui-user` when present.
`GET /api/scheduled-reports/:contractId`	`GET /scheduled-reports/:contractId`	**Loads the detail view for one scheduled report.** Includes profile, backing contract, and sorted run history.	Forwards `x-ui-user` when present.
`POST /api/scheduled-reports/:contractId/status`	`POST /scheduled-reports/:contractId/status`	**Pauses or reactivates a scheduled report.** This is the route behind the pause/activate button and was the recent missing proxy bug.	Forwards JSON body and `x-ui-user`.

3. DATABASE AND GOVERNANCE

The proxy surface for source setup, governance, and safe querying

This is the largest proxy cluster. It supports the connection wizard, allowlist governance, cataloging, business context capture, safe query execution, and the edit-connected-source view.

Database connection and governance proxies

These routes sit behind the Source, Governance, and Activate steps on the Connect page.

12 ROUTES

WEB ROUTE	UPSTREAM API	EXPLANATION	CONTEXT
`GET /api/db/context`	`GET /connections/active`	**Returns the active governed connection context.** Used to show what source is currently connected.	Forwards `x-ui-user`.
`GET /api/db/tables`	`GET /connections/tables`	**Lists available tables for selection and governance.** Used while building the allowlist.	Forwards `x-ui-user`.
`POST /api/db/test`	`POST /connections/test`	**Tests a database connection without activating it.** Used in Step A2 before connect.	Forwards JSON body and `x-ui-user`.
`POST /api/db/connect`	`POST /connections/connect`	**Creates or activates the source connection for the user.** This is the handoff from source setup into governance.	Forwards JSON body and `x-ui-user`.
`POST /api/db/allowlist`	`POST /connections/allowlist`	**Saves the governed allowlist for the connected source.** This is the key safety boundary for what downstream queries may touch.	Forwards JSON body and `x-ui-user`.
`POST /api/db/validate`	`POST /connections/validate`	**Runs validation checks before activation.** The web layer always sends an empty JSON body here.	Forwards `x-ui-user`. Body is forced to `{}`.

WEB ROUTE	UPSTREAM API	EXPLANATION	CONTEXT
`POST /api/db/fix-script`	`POST /connections/fix-script`	**Requests a provider-specific fix script.** Used when the connection needs remediation help.	Forwards JSON body and `x-ui-user`.
`GET /api/db/query-logs`	`GET /connections/query-logs`	**Fetches the governed query history.** Useful for inspection and audit-oriented screens.	Forwards `x-ui-user`.
`POST /api/db/query`	`POST /connections/query`	**Runs a governed read-only query.** Used for safe query testing, activation checks, and other preview actions.	Forwards JSON body and `x-ui-user`.
`GET /api/db/catalog`	`GET /connections/catalog`	**Returns the cataloged table and column metadata.** Chat, scoping, and config flows all depend on this.	Forwards `x-ui-user`.
`POST /api/db/catalog/refresh`	`POST /connections/catalog/refresh`	**Forces a catalog refresh.** Used when the user wants updated metadata after schema changes.	Forwards `x-ui-user`.
`POST /api/db/catalogue`	`POST /connections/catalogue`	**Starts cataloging after governance is saved.** This is the route behind the cataloging loading popup in the connect flow.	Forwards `x-ui-user`.

Business context and disconnect proxies

Small but important routes used after a connection is governed.

2 ROUTES

WEB ROUTE	UPSTREAM API	EXPLANATION	CONTEXT
`POST /api/db/business-context`	`POST /connections/business-context`	**Saves business context at the connection layer.** This feeds planning, metric interpretation, and better assumptions in scope clarification.	Forwards JSON body and `x-ui-user`.

WEB ROUTE	UPSTREAM API	EXPLANATION	CONTEXT
`POST /api/db/disconnect`	`POST /connections/disconnect`	**Disconnects the governed source for the user.** This powers the disconnect action from the edit-connected-databases view.	Forwards `x-ui-user`.

4. CONFIG AND NON-PROXY ROUTES

Configuration proxies and the few routes that bypass the shared proxy helper

Not everything under `/api` uses `proxyToApi`. The web app uses direct API-client calls for a small number of download and status routes where direct response handling is more convenient.

Configuration proxies

Used by the Config page and by metric-definition and business-context settings flows.

4 ROUTES

WEB ROUTE	UPSTREAM API	EXPLANATION	CONTEXT
`GET /api/config/global`	`GET /config/global`	**Loads the workspace-wide global config.** This is not tied to one specific user in the proxy layer.	No `x-ui-user` forwarding here.
`PUT /api/config/global`	`PUT /config/global`	**Saves the workspace-wide global config.** Used for shared metric and report consistency settings.	Forwards JSON body. No `x-ui-user`.
`GET /api/config/user-settings`	`GET /config/user-settings`	**Loads per-user settings.** This is where user-specific metric definitions and business context can live.	Forwards `x-ui-user`.
`PUT /api/config/user-settings`	`PUT /config/user-settings`	**Saves per-user settings.** This is the user-scoped counterpart to the global config routes.	Forwards JSON body and `x-ui-user`.

Direct route: run status

`GET /api/run-status/:runId` uses the web API client directly, not `proxyToApi`. It calls `apiClient.getRunStatus(runId)` and returns the payload or a `502`.

Direct route: report downloads

`GET /api/runs/:runId/pdf` and `GET /api/runs/:runId/html` also bypass the shared proxy helper. They use the web API client directly so the web app can preserve response headers and send PDF bytes or HTML text cleanly.

Why this distinction matters: if a bug affects a route under `/api/*`, the first debugging question is whether it uses the shared proxy helper or the direct web API client. The scheduled report pause bug was a shared-proxy omission. PDF and HTML fetch problems would live elsewhere.

5. ROUTE-BY-ROUTE SUMMARY

One compact table you can scan quickly on Wednesday

This is the shortest possible summary: the browser route, the upstream API route, and the job it does.

Complete proxy inventory

The entire current `proxyToApi` surface in one place.

24 ROUTES

WEB ROUTE	API ROUTE	PURPOSE
`GET /api/chat/sessions`	`/ui/chat-sessions`	List saved chat sessions.
`PUT /api/chat/sessions/:id`	`/ui/chat-sessions/:id`	Update one chat session.
`GET /api/runs/:runId/schedule-draft`	`/report-runs/:runId/schedule-draft`	Draft the schedule modal.
`POST /api/runs/:runId/schedule-profile`	`/report-runs/:runId/schedule-profile`	Save a scheduled report profile.
`GET /api/scheduled-reports`	`/scheduled-reports`	List scheduled reports.
`GET /api/scheduled-reports/:contractId`	`/scheduled-reports/:contractId`	Load one scheduled report detail view.
`POST /api/scheduled-reports/:contractId/status`	`/scheduled-reports/:contractId/status`	Pause or activate a scheduled report.
`GET /api/db/context`	`/connections/active`	Fetch active connection context.
`GET /api/db/tables`	`/connections/tables`	Fetch tables for allowlisting.
`POST /api/db/test`	`/connections/test`	Test a DB connection.
`POST /api/db/connect`	`/connections/connect`	Connect a source.

WEB ROUTE	API ROUTE	PURPOSE
`POST /api/db/allowlist`	`/connections/allowlist`	Save the governed allowlist.
`POST /api/db/validate`	`/connections/validate`	Validate before activation.
`POST /api/db/fix-script`	`/connections/fix-script`	Request a remediation script.
`GET /api/db/query-logs`	`/connections/query-logs`	Fetch governed query logs.
`POST /api/db/query`	`/connections/query`	Run a governed read-only query.
`GET /api/db/catalog`	`/connections/catalog`	Load catalog metadata.
`POST /api/db/catalog/refresh`	`/connections/catalog/refresh`	Refresh the catalog.
`POST /api/db/catalogue`	`/connections/catalogue`	Kick off cataloging after allowlist save.
`POST /api/db/business-context`	`/connections/business-context`	Save DB-level business context.
`POST /api/db/disconnect`	`/connections/disconnect`	Disconnect the source.
`GET /api/config/global`	`/config/global`	Read workspace config.
`PUT /api/config/global`	`/config/global`	Write workspace config.
`GET /api/config/user-settings`	`/config/user-settings`	Read user-scoped settings.
`PUT /api/config/user-settings`	`/config/user-settings`	Write user-scoped settings.

Reference source: `apps/web/src/app.ts`, current as of March 23, 2026.